Brute Force Algorithm for Set Cover

Problem Setup

This brute force algorithm covers the set cover problem. The main variables in the program are:

- *universe* the set of all elements that must be covered, {1, 2, …, n}.
- *subsets* the list of the m available subsets. Each subset contains some of the elements of the universe.
- *m* is the number of subsets and *n* is the number of elements in the universe.

The main goal is to select the fewest subsets whose union equals the universe. The brute force algorithm (exhaustive search) tries every possible selection of subsets, first starting with the smallest selection.

Brute Force Algorithm

```
from itertools import combinations
import time

def brute_force_set_cover(universe, subsets):
    m = len(subsets)

    for k in range(m + 1):
        for indices in combinations(range(m), k):
            covered = set()

            for i in indices:
                covered.update(subsets[i])

            if covered == universe:
                return list(indices), k

    return None, None
```

The Loops

The outer loop checks each solution in increasing order of size:

- *k = 0* – select no subsets.
- *k = 1* – select one subset.
- *k = 2* – select two subsets.
- Continue until *k = m*.

The inner loop uses *combinations(range(m), k)* to generate every combination that contains exactly *k* subsets.

Building the Union
covered becomes an empty set every new combination. The loop over *indices* then adds the elements of each selected subset with *covered.update(subsets[i])*, so that covered becomes the union of all the selected subsets.

Valid cover
if covered == universe, is the selected subsets that contain every element, so in turn the combination is a valid cover. The function then returns the list of chosen subset indices and the cover size *k*.

Optimal Solution
The outer loop tests sizes in increasing order of k. When the algorithm reaches size k, it has already tested every combination with fewer than k subsets and none of them covered the universe. Therefore, the first valid cover found must use the minimum possible number of subsets.

Timing the Program

```
U,S =
load_sts("/Users/jayden/Desktop/MLDM/Semester_1/Advanced_Algorithms/
Projects/Benchmarks/data.9")

start = time.perf_counter()     # starts measuring execution time

indices, size = brute_force_set_cover(U, S)     # runs the brute
force algorithm

end = time.perf_counter()       # stops measuing execution time

print("Selected subsets:")      # displays results

for i in indices:
    print(f"S{i + 1}: {sorted(S[i])}")

print("Minimum cover size:", size)
print(f"Execution time: {end - start:.6f} seconds")
```

time.perf_counter() is called before and after the algorithm. This is so that the execution time only covers the search and not the loading of the file. Subsets are printed as S1, S2, …, etc to match the numbering in the data file.
Results
Running the program on the STS9 benchmark gives the following:

```
Selected subsets:
S1: [2, 3, 7, 10]
```

```
S2: [1, 3, 8, 11]
S3: [1, 2, 9, 12]
S4: [1, 5, 6, 10]
S5: [2, 4, 6, 11]
Minimum cover size: 5
Execution time: 0.000078 seconds
```

The union of these subsets is {1, 2, …, 12}, so every element of the universe is covered. Because the algorithm tests all the smaller selection first, no cover with 4 or fewer subsets exist. The execution time also depends on the machine can vary between runs.

Complexity

In the worst case, the algorithm examines all 2^m combinations of subsets and building the union for each one takes time depending on the size of the chosen subset. This can lead to running time growing exponentially with *m.* Therefore, brute force is only practical for small instances as STS9.